THREAT-INTELLIGENCE KILL-CHAIN CORRELATION ENGINE

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Arikela Sumanjali (2520090190)
Savulla Rithesh (2520090122)

Team Number: 33

Section: 10

Under the guidance of

Mrs. K. Chandusha

Professor, CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Mini Project Report entitled "**THREAT-INTELLIGENCE KILL-CHAIN CORRELATION ENGINE"** is a record of Bonafide work of **Arikela Sumanjali – 2520090190, Savulla Rithesh– 2520090122,** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artifical Intelligence & data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Arikela Sumanjali– 2520090190

Savulla Rithesh– 2520090122

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**THREAT-INTELLIGENCE KILL-CHAIN CORRELATION ENGINE"** is a bonafide work done and submitted by **Arikela Sumanjali – 2520090190, Savulla Rithesh– 2520090122,** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide Mrs Chandusha, Professor, CSITwhich cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Khaja Shareef (CSIT**Head of the *Department for Department for Computer Science and Information Technology and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Arikela Sumanjali– 2520090190

Savulla Rithesh– 2520090122

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	
2.	Introduction	
3.	Literature survey	
4.	Problem Statement and Objectives	
5.	Hardware and Software requirements	
6.	Implementation	
7.	Results & Discussion	
8.	Conclusion & Future Scope	
9.	References	

1.ABSTRACT

The Threat-Intelligence Kill-Chain Correlation Engine is a cybersecurity monitoring prototype that analyses security-log text, identifies configured threat indicators, assigns a severity score, maps findings to a Cyber

Kill Chain stage, and presents saved records in a user-specific web dashboard. The system is implemented as three connected components: a Java Swing desktop application for Data Structures and Algorithms (DSA) analysis, a Python FastAPI REST backend, and a React web frontend. The backend persists users and threats in SQLite through SQLAlchemy, while JWT-based authentication associates each threat with the logged-in user. The Java implementation includes case-insensitive Knuth–Morris–Pratt (KMP) pattern matching with an LPS table, breadth-first search (BFS) and depth-first search (DFS) over a directed kill-chain graph, a manually implemented min-heap priority queue, and stable merge sort for risk ordering. Risk scores range from 1 (CRITICAL) to 4 (LOW), and the furthest mapped kill-chain stage among detected patterns is selected. The dashboard displays total, active, critical, and resolved counts and refreshes periodically to show threats saved from the Java GUI. The Java self-test included in the supplied source archive was compiled and executed; all 14 checks passed. The prototype demonstrates how core DSA techniques can be combined with authenticated APIs and persistent storage to support structured threat review. It currently analyses text supplied by the user and does not ingest live security feeds automatically.

Team Name: Threat Intelligence & Cyber Kill-Chain Monitoring

Team Members:

1. [Sumanjali Arikela, 2520090190]
2. [Rithesh Savulla,2520090122]

2. INTRODUCTION

Modern information systems produce security events from applications, endpoints, network devices, and user activity. A single log entry may contain only a small clue; when reviewed in isolation, it may not show where an attack sits in its broader lifecycle. Analysts therefore need a way to inspect suspicious text, identify indicators, assign a consistent level of urgency, and organize findings in a form that can be reviewed later.

The supplied project, Threat-Intelligence Kill-Chain Correlation Engine, is an educational prototype that applies data structures and algorithms to this problem. Its Java Swing GUI accepts a log-like text entry and runs the implemented DSA routines. KMP searches for configured keywords without relying on regular expressions or built-in substring search. A directed graph models possible transitions between the seven Cyber Kill Chain stages; BFS and DFS expose different traversal orders, and BFS parent links are used to reconstruct a shortest path in the graph. A min heap prioritizes lower numerical risk scores, while merge sort orders analysed threats by score.

The system separates analysis, persistence, and visualization. The Java client communicates with a FastAPI backend over HTTP/JSON. The backend validates requests, applies JWT authentication, and stores user and threat records in SQLite. The React dashboard retrieves the authenticated user's statistics and threat list, shows stage counts and recent records, and supports status updates and deletion. The dashboard polls the API every five seconds, so records saved from the Java GUI can appear without manually reloading the page.

2.1 Scope

- Analyse manually supplied security-log text using the configured keyword patterns.
- Demonstrate KMP/LPS, graph traversal and shortest-path reconstruction, a min heap, and merge sort in Java.
- Save analysed threat records through the API and display them in the authenticated React dashboard.
- Provide registration, login, logout, per-user threat visibility, threat status changes, and deletion.

2.2 Current limitations

The included code does not implement automatic collection from live endpoints or network sensors. The inspected Java source contains KMP, BFS/DFS, a min heap, and merge sort; Trie and Rabin–Karp are mentioned in the supplied abstract/PPT but are not present in the submitted source archive. The implementation uses SQLite, although the PPT lists MongoDB. This report documents the code archive's actual implementation.

.

3. LITERATURE SURVEY

The following related frameworks and tools provide background for the project. They are not components embedded in this prototype; they are referenced to position the project's focus on attack-stage modelling, event detection, and security-log analysis.

1. Lockheed Martin Cyber Kill Chain

The Cyber Kill Chain describes an intrusion as a sequence of seven stages and encourages defenders to understand an adversary's progression rather than treating each observation as unrelated. This stage-based framing directly informs the seven-node graph used in this project. The implementation is a simplified educational graph and should not be interpreted as a complete commercial kill-chain platform.

2. MITRE ATT&CK

MITRE ATT&CK is a public knowledge base of adversary tactics and techniques based on real-world observations. It gives analysts a structured vocabulary for discussing attacker behaviour and detection opportunities. The present project uses the distinct seven-stage Cyber Kill Chain model, not an ATT&CK mapping database; ATT&CK is included as related background.

3. Wazuh Security Platform

Wazuh is an open-source security platform with SIEM and XDR capabilities, including security-log analysis, alerting, and dashboards. It represents a broader operational platform that aggregates telemetry from many sources. The student project is narrower: it analyses text entered into the Java GUI and persists the resulting record through its API.

4. Sigma Detection Rules

Sigma provides a generic rule format for describing detections in log data, allowing detection logic to be shared across compatible security tools. It demonstrates the importance of expressing indicators and detection logic in a structured way. This prototype uses a small, hard-coded set of keyword patterns rather than parsing Sigma rule files.

5. Suricata Network Security Monitoring

Suricata is an open-source network IDS/IPS and network-security-monitoring engine. It focuses on inspecting network activity and generating security events. The project does not replace a network sensor such as Suricata; instead, it demonstrates downstream text analysis and prioritization for supplied log-like inputs.

6. KMP string matching

The Knuth–Morris–Pratt algorithm preprocesses a pattern into an LPS/failure table and uses that information to avoid rechecking already matched text after a mismatch. Its linear O(n+m) time complexity makes it suitable for demonstrating efficient indicator matching. The project implements KMP manually in Java and includes tests for case-insensitive matching, overlapping matches, and absent patterns.

3.1 Research gap and project approach

Commercial security platforms provide extensive ingestion, correlation, analytics, and response capabilities. For this academic DSA project, the learning objective is to make the underlying algorithms visible and testable. The prototype connects manually implemented pattern matching, graph traversal, heap-based prioritization, and sorting to a persistent, authenticated dashboard. It demonstrates algorithmic roles clearly, while leaving automatic telemetry ingestion, richer correlation rules, and production-scale deployment as future work.

4. Problem Statement and Objectives

Problem Statement Security-log indicators can be difficult to review consistently when alerts are considered individually, and a simple list does not show risk priority or a possible stage in an attack lifecycle.

Problem Description: The project provides a small integrated application that searches user-supplied log text for configured indicators, assigns severity and kill-chain context, prioritizes records, and saves them to a dashboard tied to the authenticated user.

Objectives:

- Implement KMP string matching and LPS preprocessing manually in Java.
- Represent the seven Cyber Kill Chain stages as a directed graph and demonstrate BFS, DFS, and BFS-based shortest-path reconstruction.

- Implement a min heap to retrieve the highest-priority threat first, using score 1 as the most urgent score.
- Implement stable merge sort to order analysed threats by risk score.
- Connect the Java GUI and React frontend to a FastAPI REST backend.
- Authenticate users with JWT and bcrypt password hashing, and restrict threat operations to the current user.
- Persist users and threat records in SQLite and display threat statistics, stage counts, severity, score, and status on the dashboard.

Execute the included algorithm self-tests to verify expected behaviour

Risk and stage mapping

Detected pattern	Risk score	Severity	Mapped stage
ransomware	1	CRITICAL	Installation
exfiltration	1	CRITICAL	Actions on Objectives
malware	2	HIGH	Installation
exploit	2	HIGH	Exploitation
c2	2	HIGH	Command & Control
suspicious	3	MEDIUM	Reconnaissance
phishing	3	MEDIUM	Delivery
port scanning	4	LOW	Reconnaissance

If several patterns are found, the lowest numerical score determines the overall severity, and the furthest mapped stage (highest stage index) is selected. If no configured pattern is found, the implementation uses LOW / score 4 and Reconnaissance as defaults.

.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- 64-bit desktop or laptop computer.
- Intel Core i5 or equivalent processor (the supplied PPT's stated baseline).
- 8 GB RAM recommended for running the backend, frontend, and Java GUI together.
- Approximately 256 GB storage as a general workstation baseline; actual project files require substantially less.
- Internet connection for installing dependencies and consulting external reference material; the local application components communicate over localhost.

Software Requirements:

Component	Technology in supplied code	Purpose
Operating system	Windows or Linux	Development and execution environment
Java	JDK 17 or later recommended by README	Swing GUI and manual DSA implementations
Backend	Python, FastAPI, Uvicorn	REST API and request handling
Persistence	SQLite, SQLAlchemy	Store user and threat records
Authentication	PyJWT and bcrypt	JWT sessions and password hashing
Frontend	React 18, Vite, React Router, CSS	Login, registration, and dashboard
IDE	Visual Studio Code (recommended)	Code editing and terminal workflow
Browser	Modern web browser	Open the local React dashboard

3 Project architecture

Client / layer	Connection	Responsibility
React + Vite frontend	HTTP/JSON → FastAPI	Authentication screens, protected dashboard, stats and threat actions
Java Swing DSA GUI	HTTP/JSON → FastAPI	Log input, algorithm views, prioritization, sorting, save analysed threat
FastAPI backend	SQLAlchemy → SQLite	JWT validation, user-scoped CRUD, stats, validation and persistence
SQLite database	Accessed only by backend	Users and threats tables

6. IMPLEMENTATION

The source archive contains three main parts: backend/, frontend/, and dsa-gui/. The implementation section below focuses on four manual DSA modules, followed by the integration flow.

6.1 Module 1 — KMP and LPS pattern matching

File: dsa-gui/src/threatintel/KMP.java. The buildLPS method constructs the longest-proper-prefix/suffix table. The search method compares text and pattern case-insensitively and returns all zero-based match positions, including overlapping matches.

```
public static int[] buildLPS(String p) {
    int m = p.length();
    int[] lps = new int[m];
    int len = 0, i = 1;
    while (i < m) {
        if (lower(p.charAt(i)) == lower(p.charAt(len))) {
            lps[i++] = ++len;
        } else if (len > 0) {
            len = lps[len - 1];
        } else {
            lps[i++] = 0;
        }
    }
    return lps;
}
```

Complexity: for text length n and pattern length m, preprocessing takes O(m) time and O(m) space; searching takes O(n+m) time and O(m) auxiliary space for the LPS table.

6.2 Module 2 — Kill-chain graph, BFS and DFS

File: dsa-gui/src/threatintel/KillChainGraph.java. The graph has seven vertices: Reconnaissance, Weaponization, Delivery, Exploitation, Installation, Command & Control, and Actions on Objectives. An adjacency list represents possible transitions. BFS uses a manually allocated array queue and records parent links; DFS recursively explores as deeply as possible before backtracking.

```
public List<Integer> bfs(int start, int[] parentOut) {
    int n = STAGES.length;
    boolean[] seen = new boolean[n];
    int[] queue = new int[n];
    int head = 0, tail = 0;
    List<Integer> order = new ArrayList<>();
```

```
    for (int i = 0; i < n; i++) parentOut[i] = -1;
    seen[start] = true;
    queue[tail++] = start;
    while (head < tail) {
        int u = queue[head++];
        order.add(u);
        for (int v : adj.get(u)) {
            if (!seen[v]) {
                seen[v] = true;
                parentOut[v] = u;
                queue[tail++] = v;
            }
        }
    }
    return order;
}
```

Complexity: BFS and DFS each run in O(V+E) time and O(V) auxiliary space for V vertices and E edges. The graph is a simplified model; the traversals show graph algorithms and do not establish that a real-world attack followed a particular path.

6.3 Module 3 — Manual min-heap priority queue

File: dsa-gui/src/threatintel/PriorityQueue.java. The queue stores Threat objects in an array-backed min heap. The comparison function ranks lower riskScore values first; when scores tie, the older inserted threat is selected first. The array expands when full.

```
public void insert(Threat t) {
    if (size == heap.length) {
        Threat[] bigger = new Threat[size * 2];
        for (int i = 0; i < size; i++) bigger[i] = heap[i];
        heap = bigger;
    }
    heap[size] = t;
    int i = size++;
    while (i > 0 && less(heap[i], heap[(i - 1) / 2])) {
        swap(i, (i - 1) / 2);
        i = (i - 1) / 2;
    }
}
```

Complexity: insert O(log n), extract-min O(log n), peek O(1), and O(n) storage for n queued threats.

6.4 Module 4 — Stable merge sort

File: dsa-gui/src/threatintel/MergeSort.java. The implementation divides the array into halves, recursively sorts each half, and merges them in ascending riskScore order. The <= comparison on the left side preserves the relative order of equal-score elements, making the sort stable.

```
private static void mergeSort(Threat[] a, Threat[] tmp, int lo, int hi) {
    if (lo >= hi) return;
    int mid = (lo + hi) / 2;
    mergeSort(a, tmp, lo, mid);
    mergeSort(a, tmp, mid + 1, hi);
    int i = lo, j = mid + 1, k = lo;
    while (i <= mid && j <= hi)
        tmp[k++] = (a[i].riskScore <= a[j].riskScore) ? a[i++] : a[j++];
    while (i <= mid) tmp[k++] = a[i++];
    while (j <= hi) tmp[k++] = a[j++];
    for (k = lo; k <= hi; k++) a[k] = tmp[k];
}
```

Complexity: O(n log n) time and O(n) auxiliary space for n threats.

6.5 Backend, database, and frontend integration

- Authentication: /api/auth/register creates a user with a bcrypt password hash; /api/auth/login returns a JWT access token and user details.
- Threat API: authenticated routes create, list, retrieve, update status, delete, and compute statistics for the current user's threats.
- Data model: users store identity and password-hash fields; threats store user_id, raw_text, detected_patterns, severity, risk_score, kill_chain_stage, status, and created_at.
- Isolation: threat queries filter by the user obtained from the JWT. A second user should not see the first user's records.
- Frontend: the protected React dashboard displays total, active, critical, and resolved counts; open threats by stage; recent threat records; and actions to investigate, resolve, or delete.
- Refresh: Dashboard.jsx polls the backend every five seconds to reflect records saved through the Java GUI.

6.6 Algorithm complexity summary

Algorithm	Purpose	Time complexity	Auxiliary space
LPS construction	Preprocess one pattern	O(m)	O(m)
KMP search	Find all pattern occurrences	O(n+m)	O(m)
BFS	Level-order graph traversal / shortest path support	O(V+E)	O(V)
DFS	Depth-first graph traversal	O(V+E)	O(V)
Min heap	Insert / extract-min / peek	O(log n) / O(log n) / O(1)	O(n) total heap
Merge sort	Sort threats by risk score	O(n log n)	O(n)

7. RESULTS & DISCUSSION

The supplied Java source includes SelfTest.java, a console test harness for the implemented algorithms. The project was compiled from the uploaded archive and the self-test was executed in the report preparation environment. All 14 checks passed.

7.1 Verified algorithm self-test output

```
PASS  LPS of 'abab'
PASS  LPS of 'aabaaab'
PASS  KMP finds ransomware at 11
PASS  KMP is case-insensitive (suspicious at 0)
PASS  KMP overlapping matches
PASS  KMP no match
PASS  BFS order [0, 1, 2, 3, 4, 5, 6]
PASS  DFS order [0, 1, 3, 4, 5, 6, 2]
PASS  BFS and DFS differ
PASS  Shortest path to stage 6 [0, 1, 3, 5, 6]
PASS  Peek returns lowest score, oldest first (t1)
PASS  Heap extracts in sorted order 1112233344
PASS  Extract on empty is null
PASS  Merge sort stable ascending: bedac

ALL TESTS PASSED
```

7.2 Interpretation

- KMP/LPS: tests cover two LPS arrays, case-insensitive matching, a known match position, overlapping matches, and a no-match case.
- Graph: tests verify the BFS order, DFS order, that the two traversals differ, and the BFS-derived path to the final stage.
- Priority queue: tests confirm the minimum risk score is extracted first, older threats break ties, extracted scores are ordered, and extraction on an empty heap returns null.
- Merge sort: a test verifies stable ascending ordering by risk score.

7.3 Application workflow

1. Start the FastAPI backend and open the React application in a browser.
2. Register or log in, then open the Java Swing GUI and authenticate using the same account.
3. Load a sample log and run the complete DSA analysis to view detected patterns, severity, stage, BFS/DFS output, priority queue, and merge-sort results.
4. Save the analysed threat to the dashboard. The dashboard polls the API every five seconds and should display the record and updated statistics.
5. Use dashboard actions to mark a threat Investigating or Resolved, or delete it. A separate account should show only its own records.

Evidence note: the supplied PPT and abstract contain no application screenshots. To avoid presenting invented screenshots as captured application output, this report records the verified self-test output and describes the UI workflow. If the course evaluator strictly requires screenshots, capture the actual running dashboard and Java GUI and insert them here before submission.

8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion

The Threat-Intelligence Kill-Chain Correlation Engine demonstrates the use of core data structures and algorithms in a small cybersecurity workflow. KMP provides linear-time pattern searching; the directed

graph supports BFS/DFS exploration and BFS-based shortest-path reconstruction; a manual min heap prioritizes lower-score threats; and merge sort orders records by risk. The Java GUI communicates with a FastAPI backend, which stores user-scoped threat records in SQLite and exposes them to a React dashboard. The included algorithm test harness was compiled and run successfully, with all 14 checks passing. The result is an educational prototype for algorithm demonstration and threat-record review, not a production security monitoring system.

8.2 Future Scope

Near-term improvements

- Add screenshots and reproducible end-to-end integration tests for registration, login, save-to-dashboard, status updates, and per-user isolation.
- Improve validation and normalization of log text, and add configurable indicator lists instead of hard-coded keywords.
- Add clearer graph visualizations that explain the selected path and distinguish a modelled path from evidence of an actual attack.

Medium-term improvements

- Implement and test Trie-based indicator lookup and Rabin–Karp matching if these algorithms remain project requirements; currently they are described in the supplied abstract/PPT but are not implemented in the archive.
- Support import of log files and structured events, timestamps, source identifiers, and configurable correlation rules.
- Add pagination, search, filters, audit logging, stronger API error handling, and automated backend/frontend tests.

Long-term improvements

- Integrate with authorized SIEM, endpoint, or network-monitoring data sources and add a queue for high-volume event processing.
- Map detected behaviours to a maintained threat knowledge base and evaluate detection quality against labelled datasets.
- Assess deployment security, secret management, database migrations, monitoring, backup, and scalability before any production use.

9. REFERENCES

[1] Lockheed Martin, “Cyber Kill Chain®.” https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html

[2] MITRE, “MITRE ATT&CK.” https://www.mitre.org/focus-areas/cybersecurity/mitre-attack

[3] Wazuh, “Open Source Security Platform / SIEM.” https://wazuh.com/platform/siem/

[4] SigmaHQ, “Sigma: Generic Signature Format for SIEM Systems.” https://github.com/SigmaHQ/sigma

[5] Open Information Security Foundation, “What is Suricata?” https://docs.suricata.io/en/latest/what-is-suricata.html

[6] National Institute of Standards and Technology, “Knuth-Morris-Pratt algorithm,” Dictionary of Algorithms and Data Structures. https://xlinux.nist.gov/dads/HTML/knuthMorrisPratt.html

[7] React, “React Documentation.” https://react.dev/learn

[8] FastAPI, “FastAPI Documentation.” https://fastapi.tiangolo.com/

[9] SQLite, “SQLite Documentation.” https://www.sqlite.org/docs.html

[10] SQLAlchemy, “SQLAlchemy Documentation.” https://docs.sqlalchemy.org/